SQL Squad Assignment 5

SQL Squad Assignment 5.....1
Task 1.....1
Task 2: (E)ER-Diagram.....1
Task 3: (E)ER-diagram to Relational Mapping.....5
Step 1: Mapping of Regular Entity Types.....5
Step 2: Mapping of Weak Entity Types.....6
Step 3: Mapping of Binary 1:1 Relationships.....7
Step 4: Mapping of Binary 1:N Relationships.....8
Step 5: Mapping of Binary M:N Relationships.....9
Step 6: Mapping of multivalued attributes.....9
Step 7: Mapping of N-ary relationships.....9
Step 9: Mapping unions.....11
Final Relational Model.....11
Task 4: Relational Schema.....12
Task 5: Web-based Application.....20
Task 6: Data.....20
Task 7: Analyse and Optimise.....20
Task 8: Development.....21
Task 9: Demo.....21
Contributions to the project:.....21
Link to GitHub repository.....22

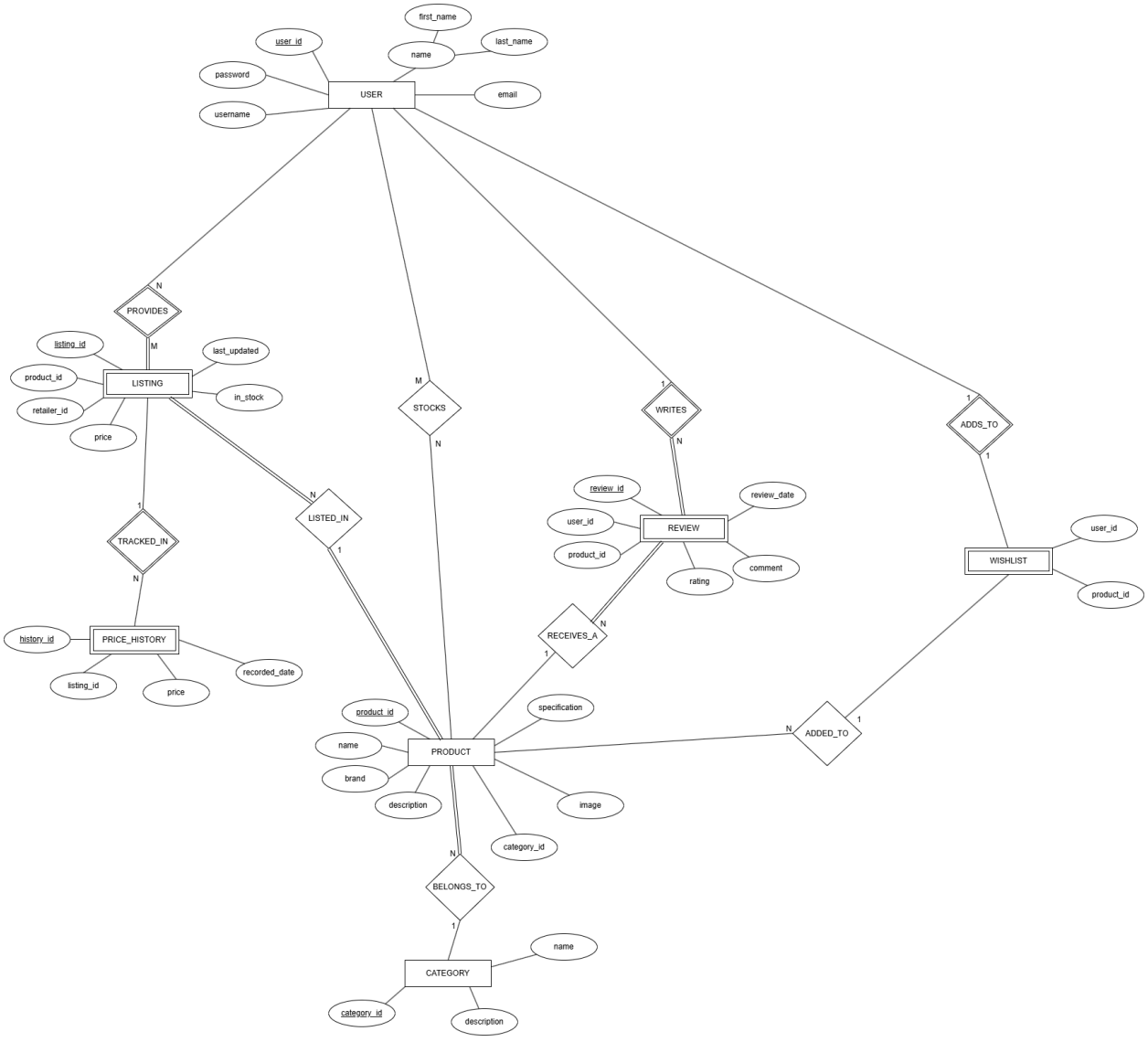
We did not use a management tool for our web-application but we did use a management tool, phpMyAdmin, for our database.

Task 1

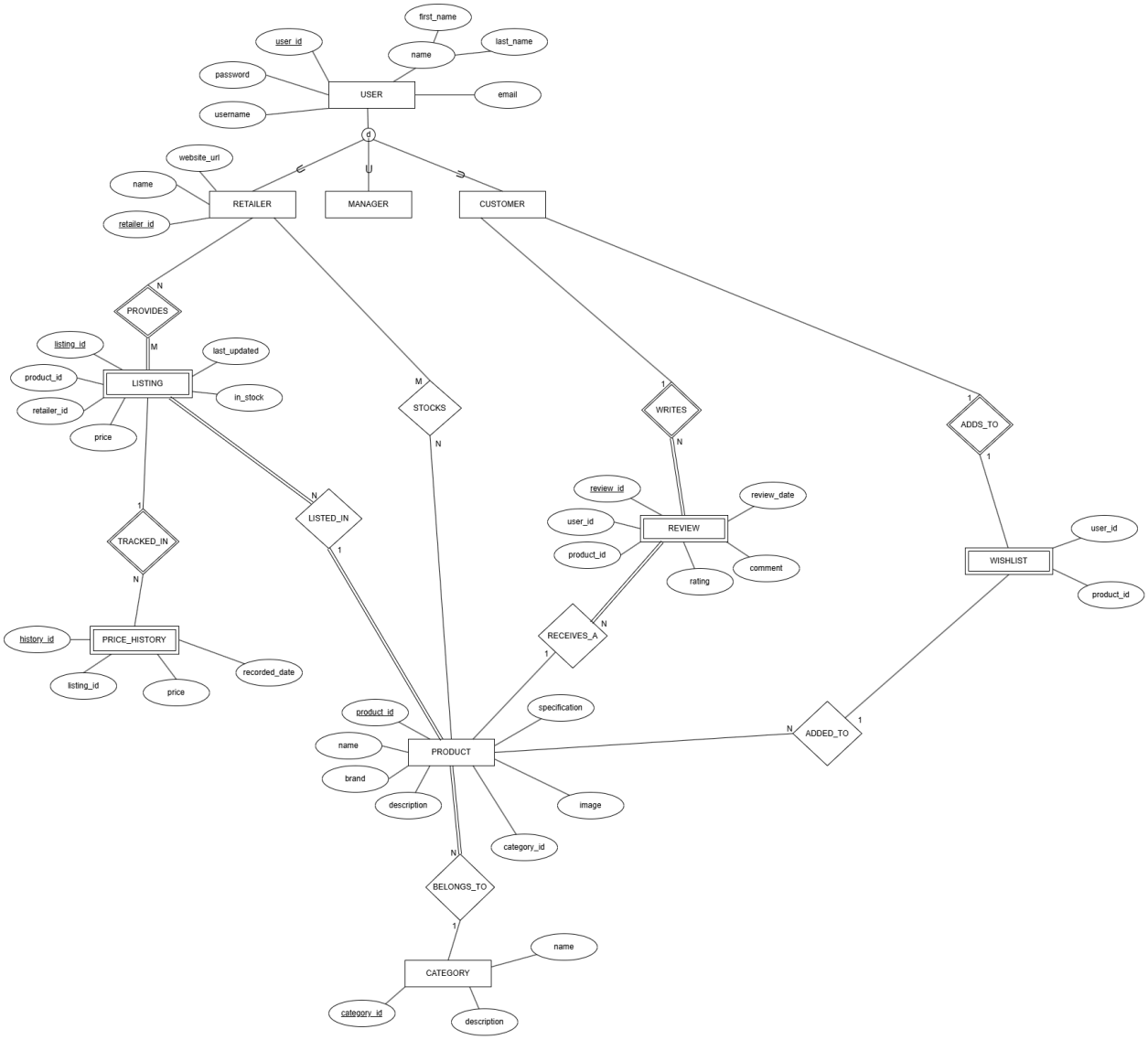
This task has already been submitted.

Task 2: (E)ER-Diagram

Iteration 1:



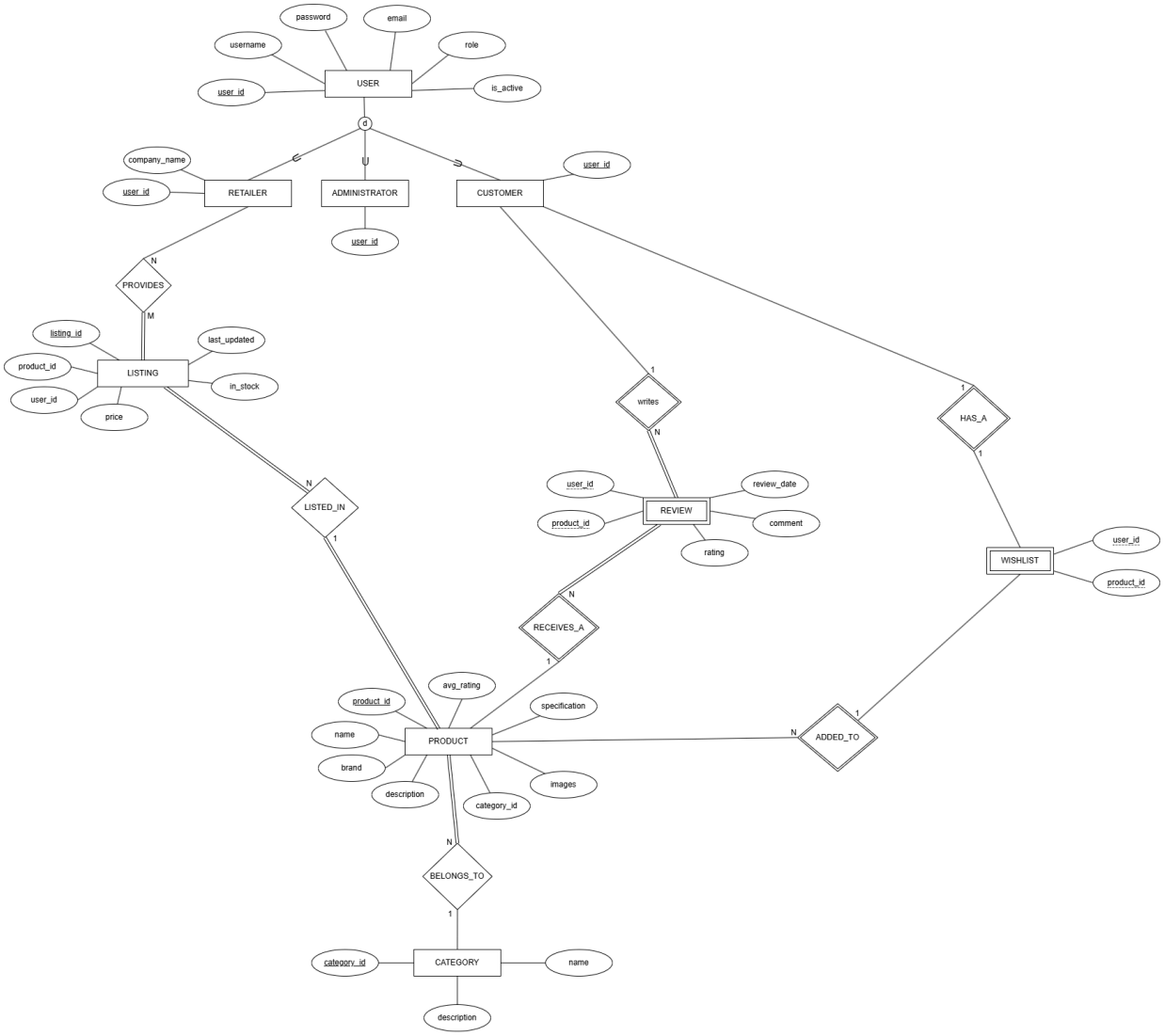
Iteration 2:



From iteration 1 to iteration 2:

- We took into account that there will be different views for different users, therefore creating a specialisation for the different user views. User is the super class and Retailer, Manager and Customer are the subclasses.

Final iteration



From iteration 2 to final iteration:

- We removed price_history as it is not necessary for our price comparison website. The website focuses on current comparisons of prices between different retailers instead of comparing previous prices of the products.
- We removed the composite name attribute, there was no need for this because our users can be identified by their username or email.
- Manager was changed to administrator. It makes more sense to have an administrator as they have control over the users and access to the website performance and analytics.
- Added the role and is_active attributes to User. The role attribute makes identifying the type of user easier, and therefore making implementation easier.
- Removed the direct relationship between Retailer and Product as they are indirectly connected via Listing. The retailer provides a listing not the physical product.
- Ensured correct entity types (strong versus weak entities), participation and cardinality.

Assumptions for the (E)ER Diagram:

- The administrators are hard coded into the database as regular users are not allowed to register as administrators. The administrator is_active attribute is always set to true.
- Due to the fact administrators are hardcoded into the database their passwords will be hashed using an online line password hasher that uses B-Crypt in order to match our password hashing method.
- When a new Retailer registers, their is_active attribute will automatically be set to false. This is to ensure Retailers are properly screened by the CompareIt staff before they are allowed access to the website and add products and listings of their own. Once they have been screened an administrator can then go and allow their account and the retailer will then be able to access the website.

Task 3: (E)ER-diagram to Relational Mapping

Step 1: Mapping of Regular Entity Types

USER

user_id	username	password	email	role	is_active

PRODUCT

product_id	name	brand	specification	description	images

CATEGORY

category_id	name	description

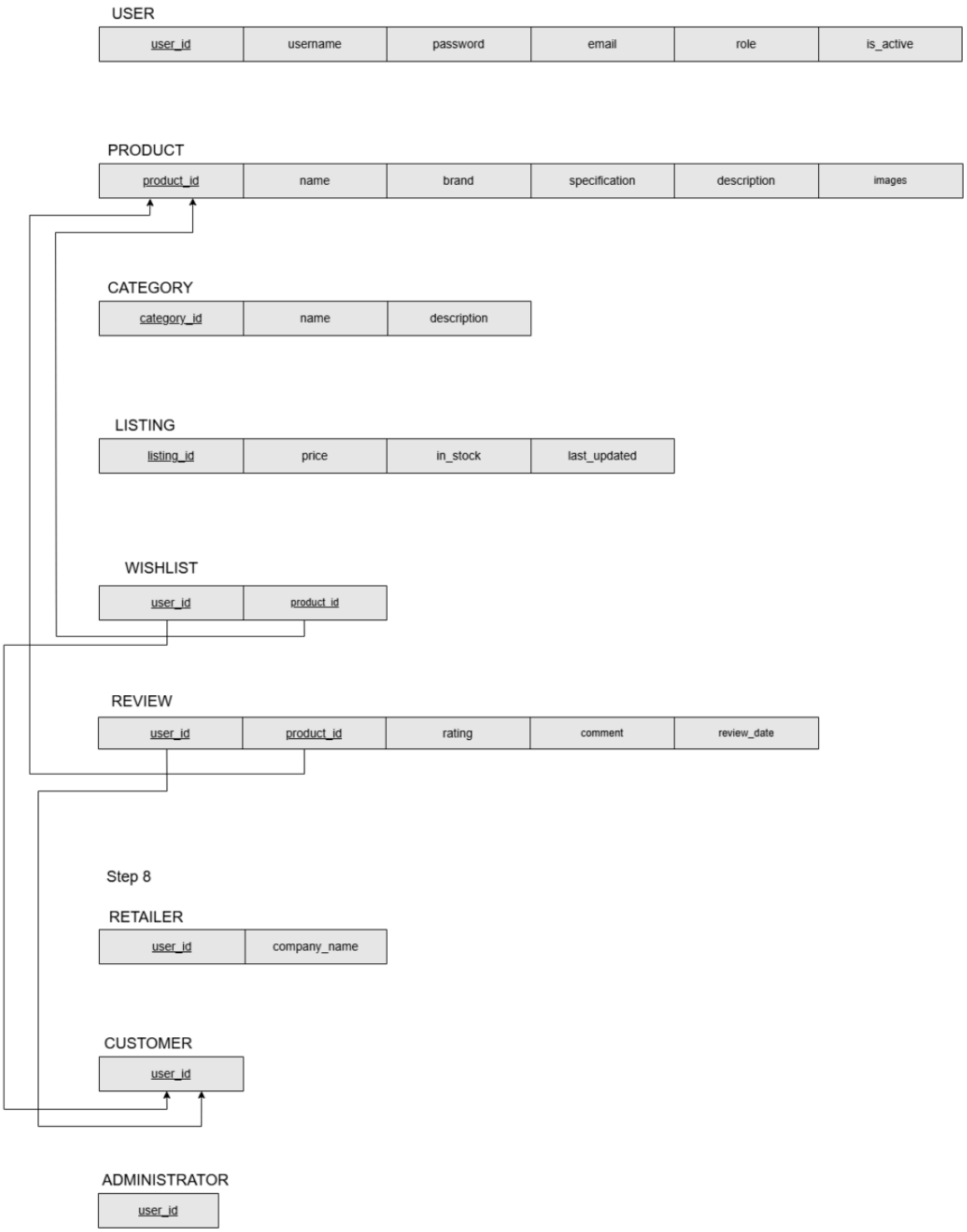
LISTING

listing_id	price	in_stock	last_updated

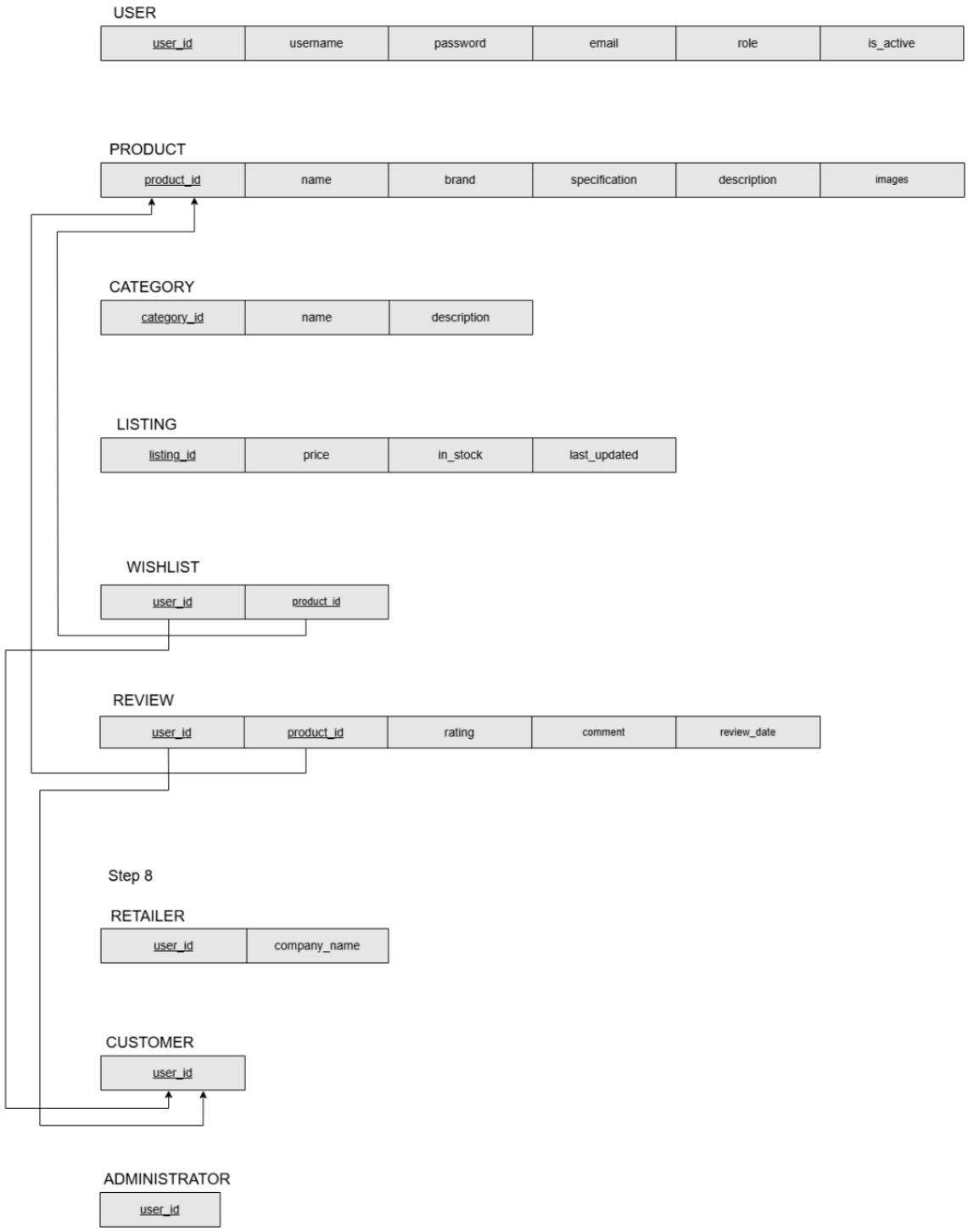
LISTING

listing_id	price	in_stock	last_updated

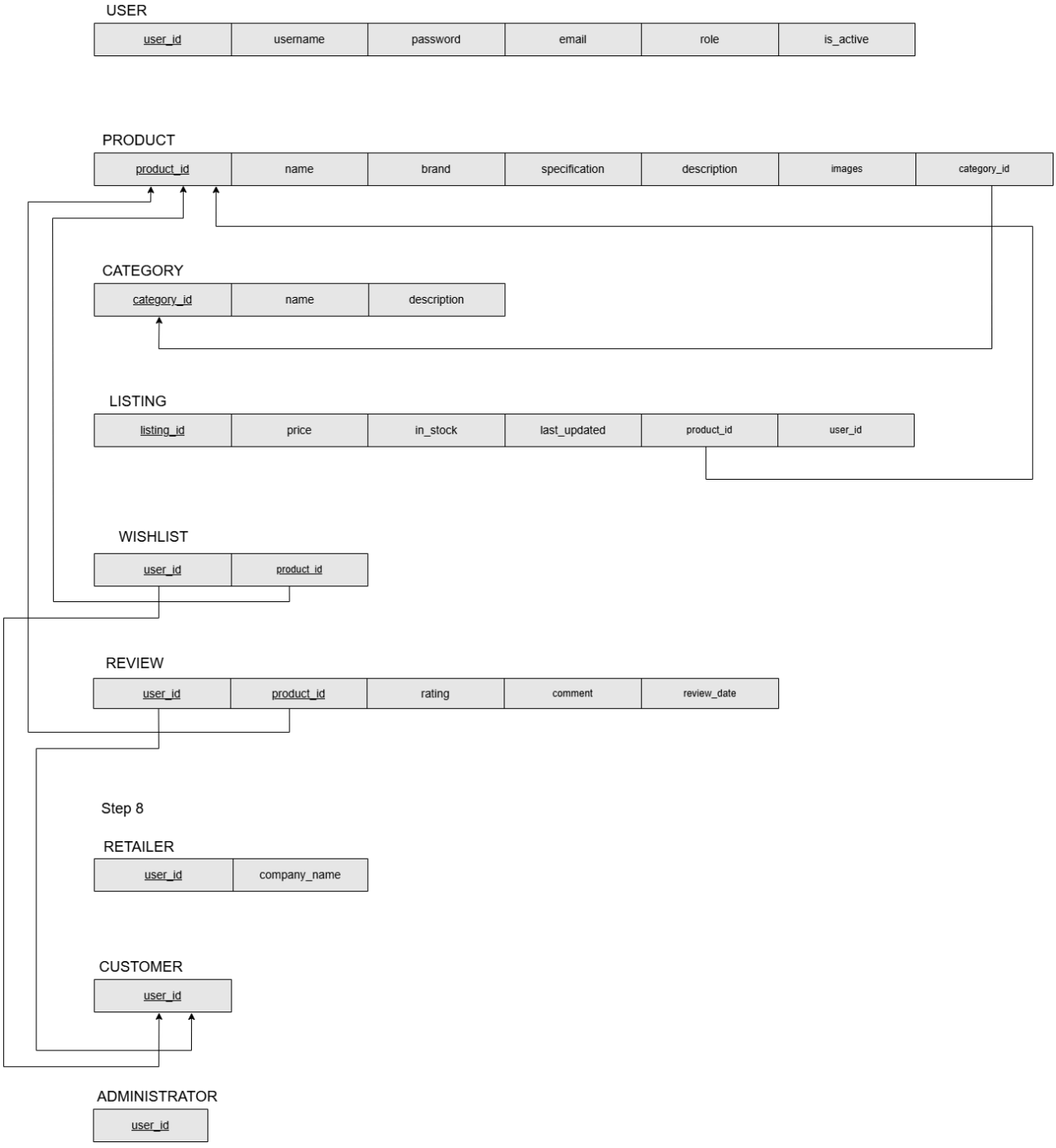
Step 2: Mapping of Weak Entity Types



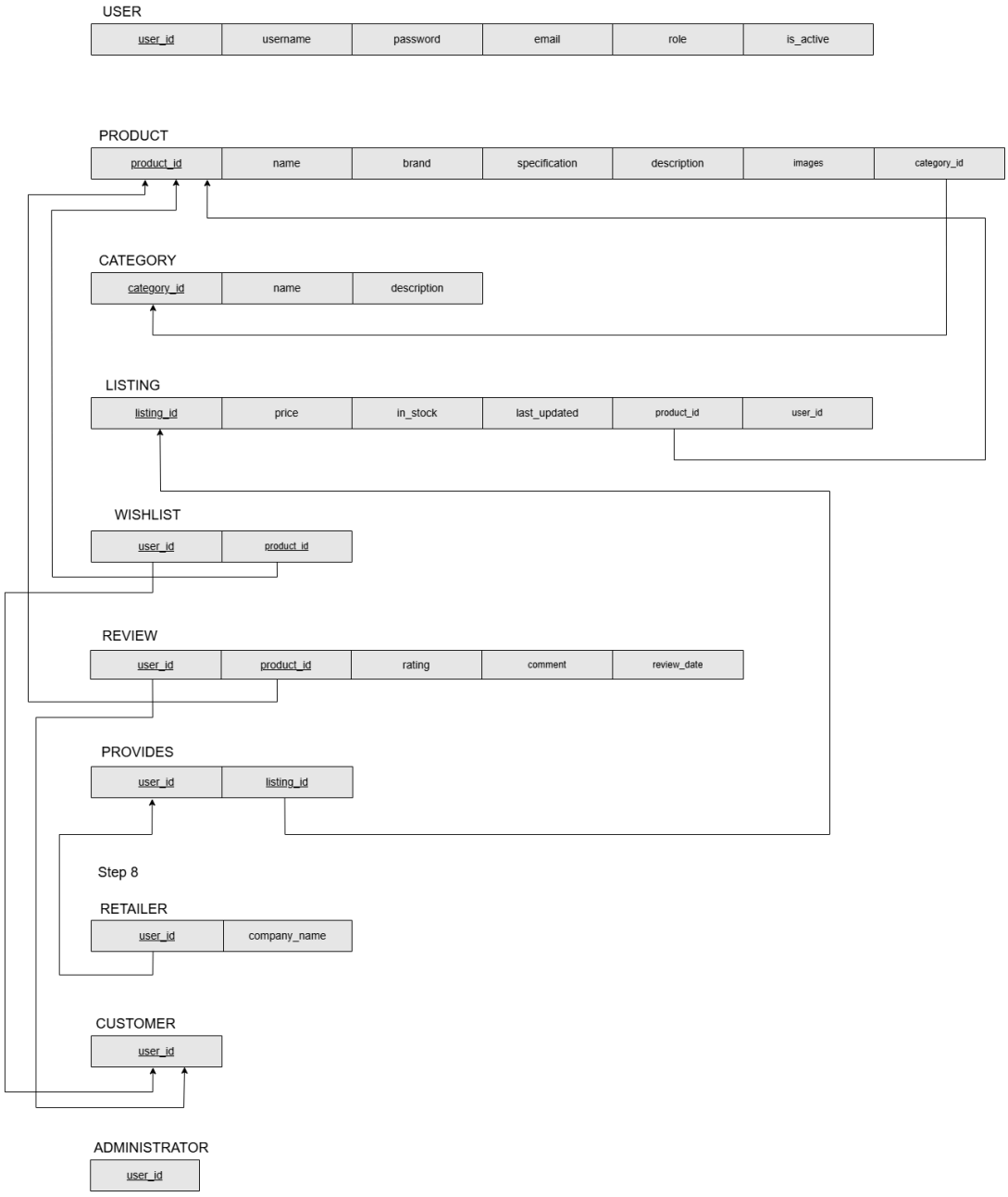
Step 3: Mapping of Binary 1:1 Relationships



Step 4: Mapping of Binary 1:N Relationships



Step 5: Mapping of Binary M:N Relationships



Step 6: Mapping of multivalued attributes

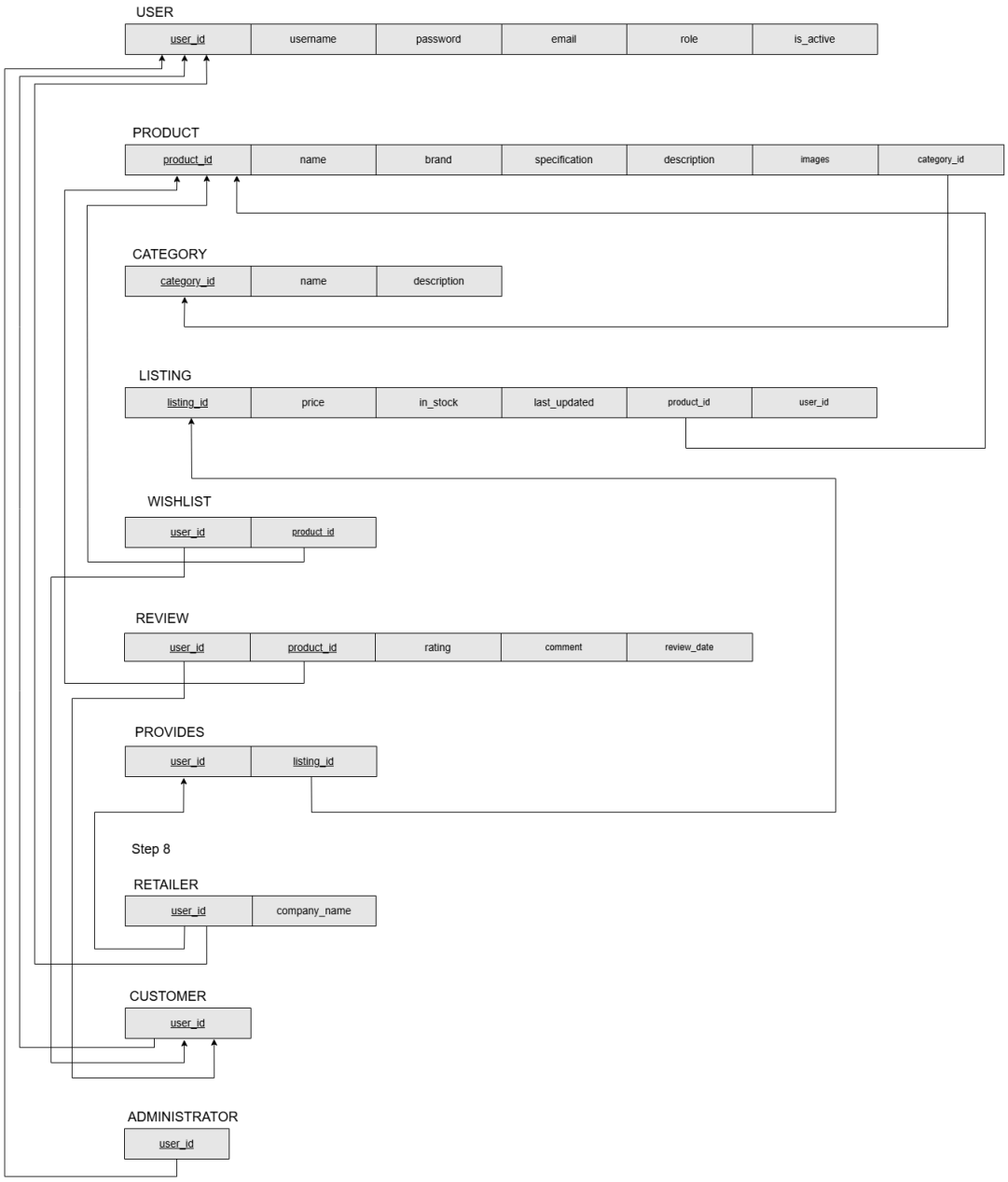
This is not applicable as we do not have any multivalued attributes.

Step 7: Mapping of N-ary relationships

This is not applicable as we do not have any N-ary relationships.

Step 8: Mapping specialisation and generalisation

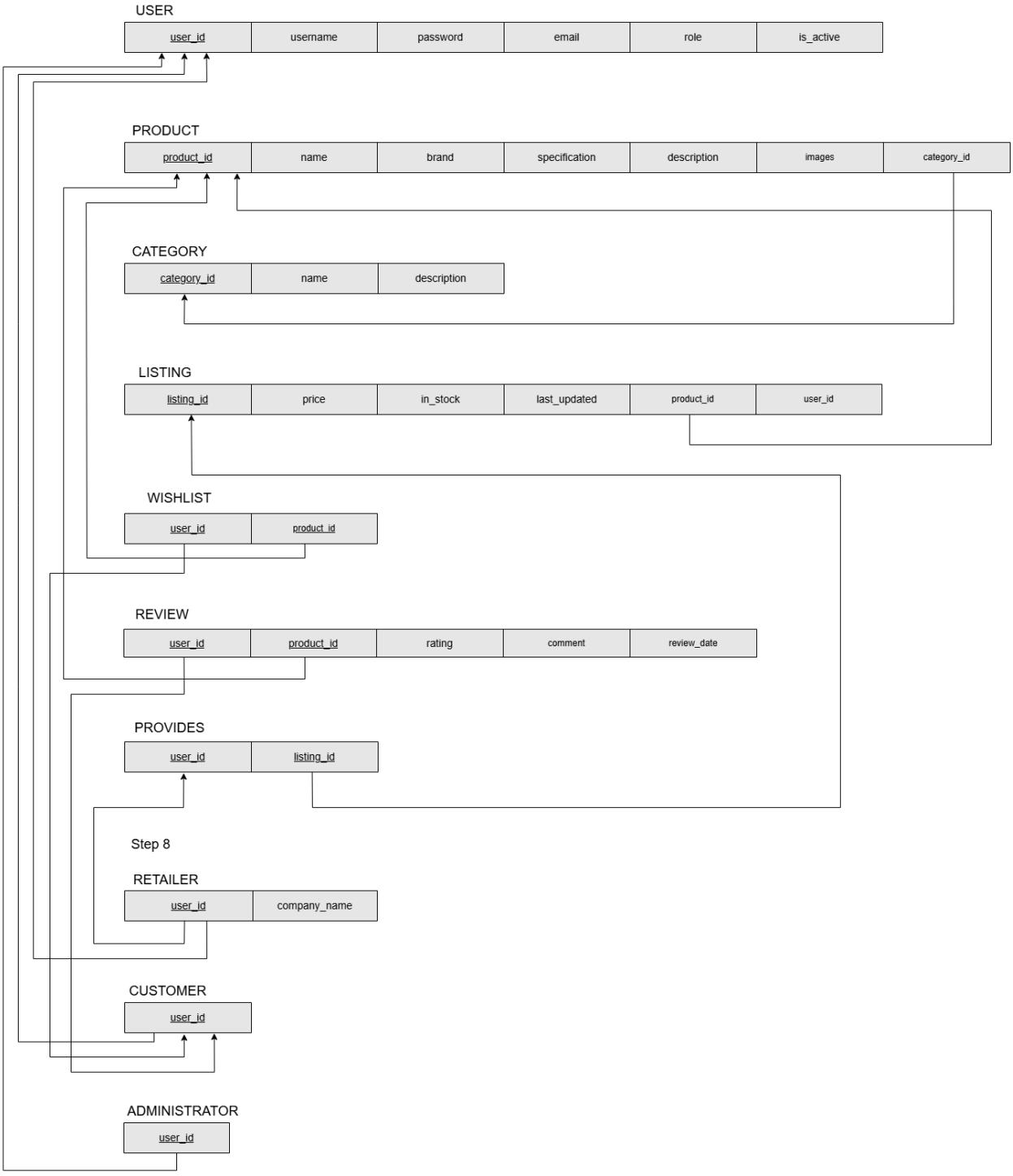
The conversion for specialisations has multiple solutions. We chose option 8A, multiple relations - superclass and subclasses, as this works well for disjoint specialisations and our specialisation is disjoint. As we do not have many unique attributes for each specialisation, this simplified the implementation.
The reason we included the specialisation is because of the relationships with other entities. For example, only a customer has a wishlist and only a retailer can provide listings.



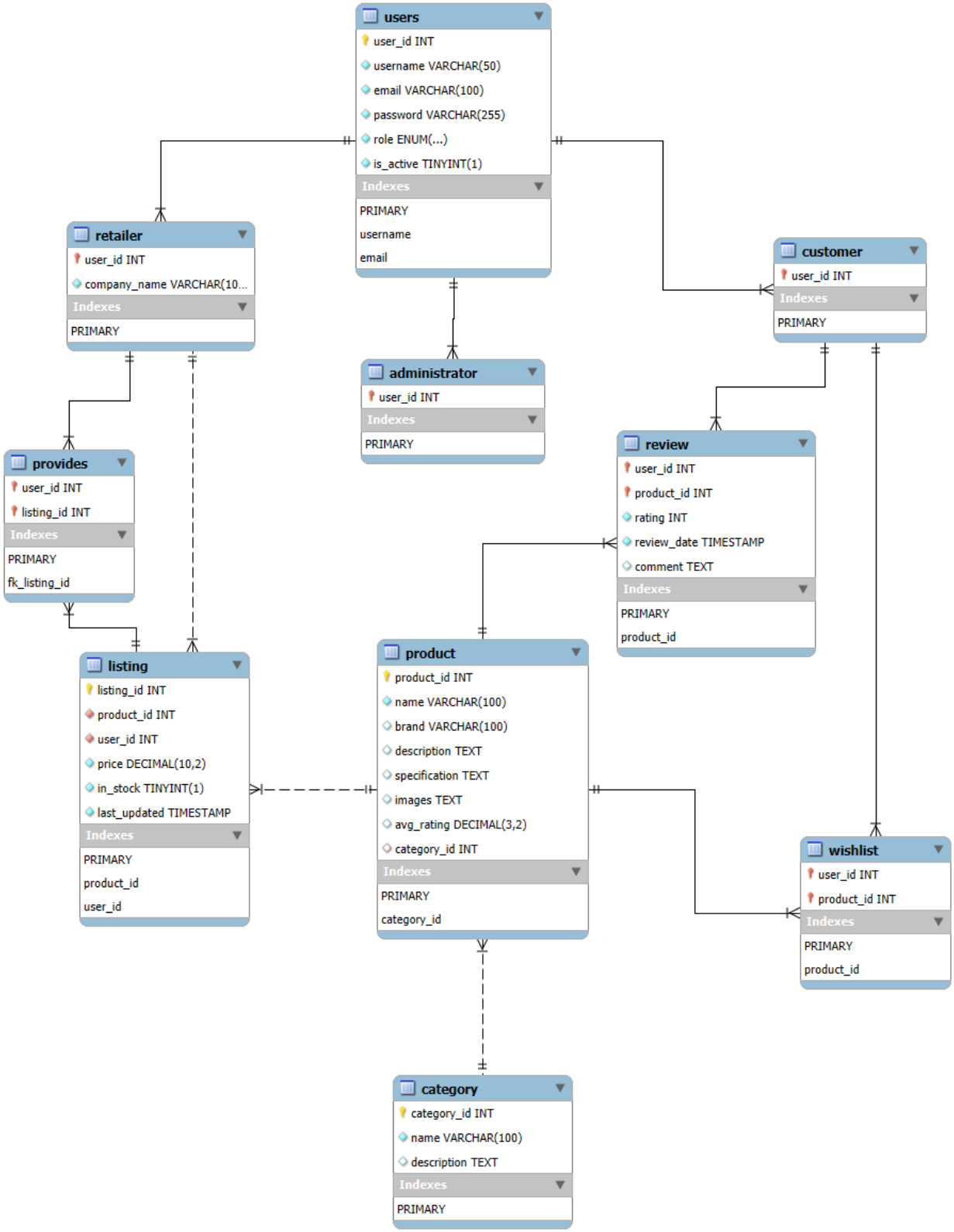
Step 9: Mapping unions

This is not applicable as we do not have any unions.

Final Relational Model



Task 4: Relational Schema



```sql
-- Database: `u24601358_SQL_Squad`
--
CREATE DATABASE IF NOT EXISTS `u24601358_SQL_Squad` DEFAULT CHARACTER SET
utf8mb4 COLLATE utf8mb4_general_ci;
USE `u24601358_SQL_Squad`;

-- --------------------------------------------------------

--
-- Table structure for table `ADMINISTRATOR`
--

CREATE TABLE `ADMINISTRATOR` (
  `user_id` int(11) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

-- --------------------------------------------------------

--
-- Table structure for table `CATEGORY`
--

CREATE TABLE `CATEGORY` (
  `category_id` int(11) NOT NULL,
  `name` varchar(100) NOT NULL,
  `description` text DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

-- --------------------------------------------------------

--
-- Table structure for table `CUSTOMER`
--

CREATE TABLE `CUSTOMER` (
  `user_id` int(11) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

-- --------------------------------------------------------

--
-- Table structure for table `LISTING`
--
```

```
CREATE TABLE `LISTING` (
  `listing_id` int(11) NOT NULL,
  `product_id` int(11) NOT NULL,
  `user_id` int(11) NOT NULL,
  `price` decimal(10,2) NOT NULL,
  `in_stock` tinyint(1) NOT NULL,
  `last_updated` timestamp NOT NULL DEFAULT current_timestamp() ON UPDATE
current_timestamp()
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

-- --------------------------------------------------------

--
-- Table structure for table `PRODUCT`
--

CREATE TABLE `PRODUCT` (
  `product_id` int(11) NOT NULL,
  `name` varchar(100) NOT NULL,
  `brand` varchar(100) DEFAULT NULL,
  `description` text DEFAULT NULL,
  `specification` text DEFAULT NULL,
  `images` text DEFAULT NULL,
  `avg_rating` decimal(3,2) DEFAULT NULL,
  `category_id` int(11) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

-- --------------------------------------------------------

--
-- Table structure for table `PROVIDES`
--

CREATE TABLE `PROVIDES` (
  `user_id` int(11) NOT NULL,
  `listing_id` int(11) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

-- --------------------------------------------------------

--
-- Table structure for table `RETAILER`
```

```
--

CREATE TABLE `RETAILER` (
  `user_id` int(11) NOT NULL,
  `company_name` varchar(100) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

-- --------------------------------------------------------

--
-- Table structure for table `REVIEW`
--

CREATE TABLE `REVIEW` (
  `user_id` int(11) NOT NULL,
  `product_id` int(11) NOT NULL,
  `rating` int(11) NOT NULL,
  `review_date` timestamp NOT NULL DEFAULT current_timestamp(),
  `comment` text DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

-- --------------------------------------------------------

--
-- Table structure for table `USERS`
--

CREATE TABLE `USERS` (
  `user_id` int(11) NOT NULL,
  `username` varchar(50) NOT NULL,
  `email` varchar(100) NOT NULL,
  `password` varchar(255) NOT NULL,
  `role` enum('customer','retailer','admin') NOT NULL,
  `is_active` tinyint(1) NOT NULL DEFAULT 1
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

-- --------------------------------------------------------

--
-- Table structure for table `WISHLIST`
--

CREATE TABLE `WISHLIST` (
```

```
  `user_id` int(11) NOT NULL,
  `product_id` int(11) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

--
-- Indexes for dumped tables
--

--
-- Indexes for table `ADMINISTRATOR`
--
ALTER TABLE `ADMINISTRATOR`
  ADD PRIMARY KEY (`user_id`);

--
-- Indexes for table `CATEGORY`
--
ALTER TABLE `CATEGORY`
  ADD PRIMARY KEY (`category_id`);

--
-- Indexes for table `CUSTOMER`
--
ALTER TABLE `CUSTOMER`
  ADD PRIMARY KEY (`user_id`);

--
-- Indexes for table `LISTING`
--
ALTER TABLE `LISTING`
  ADD PRIMARY KEY (`listing_id`),
  ADD KEY `product_id` (`product_id`),
  ADD KEY `user_id` (`user_id`);

--
-- Indexes for table `PRODUCT`
--
ALTER TABLE `PRODUCT`
  ADD PRIMARY KEY (`product_id`),
  ADD KEY `category_id` (`category_id`);

--
-- Indexes for table `PROVIDES`
```

```
--
ALTER TABLE `PROVIDES`
  ADD PRIMARY KEY (`user_id`,`listing_id`),
  ADD KEY `fk_listing_id` (`listing_id`);

--
-- Indexes for table `RETAILER`
--
ALTER TABLE `RETAILER`
  ADD PRIMARY KEY (`user_id`);

--
-- Indexes for table `REVIEW`
--
ALTER TABLE `REVIEW`
  ADD PRIMARY KEY (`user_id`,`product_id`),
  ADD KEY `product_id` (`product_id`);

--
-- Indexes for table `USERS`
--
ALTER TABLE `USERS`
  ADD PRIMARY KEY (`user_id`),
  ADD UNIQUE KEY `username` (`username`),
  ADD UNIQUE KEY `email` (`email`);

--
-- Indexes for table `WISHLIST`
--
ALTER TABLE `WISHLIST`
  ADD PRIMARY KEY (`user_id`,`product_id`),
  ADD KEY `product_id` (`product_id`);

--
-- AUTO_INCREMENT for dumped tables
--


--
-- AUTO_INCREMENT for table `CATEGORY`
--
ALTER TABLE `CATEGORY`
  MODIFY `category_id` int(11) NOT NULL AUTO_INCREMENT;
```

```
--
-- AUTO_INCREMENT for table `LISTING`
--
ALTER TABLE `LISTING`
  MODIFY `listing_id` int(11) NOT NULL AUTO_INCREMENT;

--
-- AUTO_INCREMENT for table `PRODUCT`
--
ALTER TABLE `PRODUCT`
  MODIFY `product_id` int(11) NOT NULL AUTO_INCREMENT;

--
-- AUTO_INCREMENT for table `USERS`
--
ALTER TABLE `USERS`
  MODIFY `user_id` int(11) NOT NULL AUTO_INCREMENT;

--
-- Constraints for dumped tables
--

--
-- Constraints for table `ADMINISTRATOR`
--
ALTER TABLE `ADMINISTRATOR`
  ADD CONSTRAINT `ADMINISTRATOR_ibfk_1` FOREIGN KEY (`user_id`) REFERENCES
`USERS` (`user_id`) ON DELETE CASCADE;

--
-- Constraints for table `CUSTOMER`
--
ALTER TABLE `CUSTOMER`
  ADD CONSTRAINT `CUSTOMER_ibfk_1` FOREIGN KEY (`user_id`) REFERENCES
`USERS` (`user_id`) ON DELETE CASCADE;

--
-- Constraints for table `LISTING`
--
ALTER TABLE `LISTING`
  ADD CONSTRAINT `LISTING_ibfk_1` FOREIGN KEY (`product_id`) REFERENCES
`PRODUCT` (`product_id`) ON DELETE CASCADE,
  ADD CONSTRAINT `LISTING_ibfk_2` FOREIGN KEY (`user_id`) REFERENCES
```

```
`RETAILER` (`user_id`) ON DELETE CASCADE;

--
-- Constraints for table `PRODUCT`
--
ALTER TABLE `PRODUCT`
  ADD CONSTRAINT `PRODUCT_ibfk_1` FOREIGN KEY (`category_id`) REFERENCES
`CATEGORY` (`category_id`) ON DELETE SET NULL;

--
-- Constraints for table `PROVIDES`
--
ALTER TABLE `PROVIDES`
  ADD CONSTRAINT `fk_listing_id` FOREIGN KEY (`listing_id`) REFERENCES
`LISTING` (`listing_id`) ON DELETE CASCADE ON UPDATE CASCADE,
  ADD CONSTRAINT `fk_user_retailer` FOREIGN KEY (`user_id`) REFERENCES
`RETAILER` (`user_id`) ON DELETE CASCADE ON UPDATE CASCADE;

--
-- Constraints for table `RETAILER`
--
ALTER TABLE `RETAILER`
  ADD CONSTRAINT `RETAILER_ibfk_1` FOREIGN KEY (`user_id`) REFERENCES
`USERS` (`user_id`) ON DELETE CASCADE;

--
-- Constraints for table `REVIEW`
--
ALTER TABLE `REVIEW`
  ADD CONSTRAINT `REVIEW_ibfk_1` FOREIGN KEY (`user_id`) REFERENCES
`CUSTOMER` (`user_id`) ON DELETE CASCADE,
  ADD CONSTRAINT `REVIEW_ibfk_2` FOREIGN KEY (`product_id`) REFERENCES
`PRODUCT` (`product_id`) ON DELETE CASCADE;

--
-- Constraints for table `WISHLIST`
--
ALTER TABLE `WISHLIST`
  ADD CONSTRAINT `WISHLIST_ibfk_1` FOREIGN KEY (`user_id`) REFERENCES
`CUSTOMER` (`user_id`) ON DELETE CASCADE,
  ADD CONSTRAINT `WISHLIST_ibfk_2` FOREIGN KEY (`product_id`) REFERENCES
`PRODUCT` (`product_id`) ON DELETE CASCADE;
COMMIT;
```

Task 5: Web-based Application

The web-based application file has been included in the uploaded zip file.

Task 6: Data

We chose to populate our database by hand. This method worked well for us as our website focuses on a niche (make-up products), it allowed us to have full control over our attributes and get exactly what we wanted, making it easier to have a good idea of our desired output. Although we have a substantial amount of data, it is still a small data set and it allows us to test our website effectively. The process of populating was simple as we split the insertion of data amongst us.

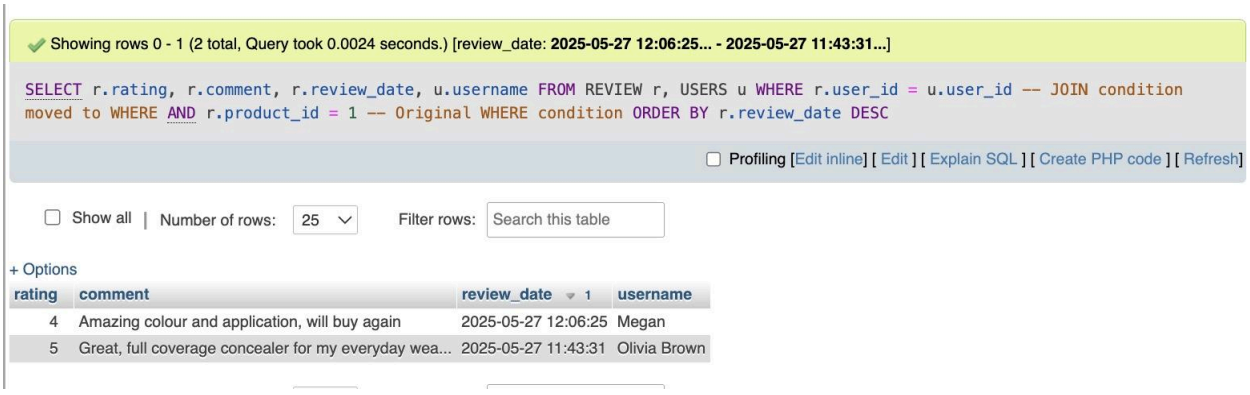
Task 7: Analyse and Optimise

The query we optimised was: getProductReviews

The original query used the cartesian product which took 0,0024 seconds to execute. The optimised version used a join which made the execution time 0.0019 seconds. Both return the same results.

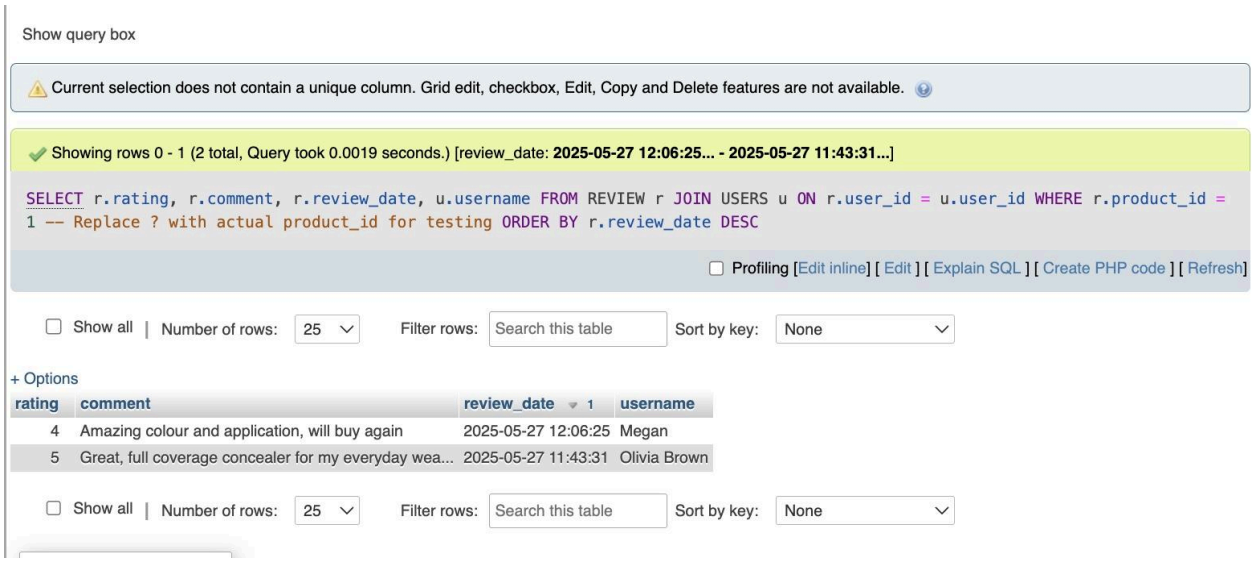
The gain in performance was observed as a cartesian product creates every possible combination of rows between the tables where a join only combines rows that meet the join condition and processes the rows that only satisfy the condition. The cartesian product generates more intermediate rows which makes more filtering, which means more SELECTs will be used, which therefore takes longer processing time. Therefore the join is more efficient and you can see the shorter processing time in the query execution.

Original query before optimisation



SQL Squad: u24711013, u24601358, u24575292, u24647374, u24593240, u24713122

Query after optimisation

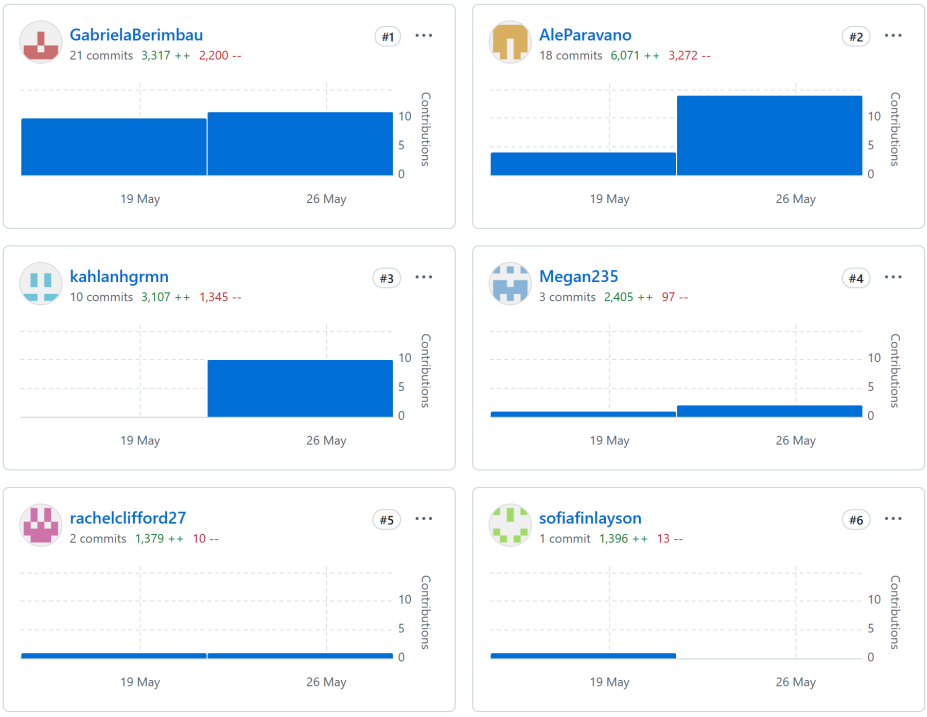


Task 8: Development

Will be shown in the demo.

Task 9: Demo

Contributions to the project:



Gabriela - (E)ER diagram, relational mapping, insertion of data into the database, retailerView javascript and corresponding API endpoints and compilation of the pdf document.

Kahlan - (E)ER diagram, creation of database and tables, insertion of data into the database, Admin page Javascript and corresponding API endpoints with "User Management", "Analytics" and "Review Management" sections.

Megan - (E)ER diagram, relational mapping, insertion of data into the database, View page functionality with API endpoints, Top Rated page functionality with API endpoints.

Rachel - (E)ER diagram, insertion of data into the database, front-end PHP and CSS, and demo presentation.

Sofia - (E)ER diagram, insertion of data into the database, front-end PHP and CSS, and demo presentation.

Alessandro - Insertion of data into the database. The login, signup, index (home), products and wishlist pages API endpoints, Javascript and additional CSS. Additional protection implementation.

Link to GitHub repository

https://github.com/GabrielaBerimbau/SQL_Squad